CS 341 Algorithms 期中考试复习总结

Lecture 1–6 · Lab 1–3 · Homework 1 浓缩笔记

范围：渐近分析 / 解递归式 / 分治 / BFS / DFS ｜ ★ = 期中高频考点

使用说明：本笔记按讲义顺序组织。每章含「知识点梳理」「核心公式」「典型例题与证明思路」「易错提醒」。文末有综合速查表。英文术语保留原文，括号附中文。公式符号与幻灯片一致。

第一章　引言：渐近分析与 3SUM（L01）

知识点梳理

渐近记号（asymptotic notation）★

设 $f(n), g(n)$ 为两个函数，用极限定义（★ 必背）：

- 上界（upper bound, big-O）：$g(n) = O(f(n))$ 若 $\lim_{n\to\infty} \frac{g(n)}{f(n)} \le c$，对某个与 n 无关的常数 c
- 下界（lower bound, big-Ω）：$g(n) = \Omega(f(n))$ 若 $\lim_{n\to\infty} \frac{g(n)}{f(n)} \ge c$
- 同阶（same order, big-Θ）：$g(n) = \Theta(f(n))$ 若 $\lim_{n\to\infty} \frac{g(n)}{f(n)} = c$
- 松上界（loose upper bound, small-o）：$g(n) = o(f(n))$ 若 $\lim_{n\to\infty} \frac{g(n)}{f(n)} = 0$
- 松下界（loose lower bound, small-ω）：$g(n) = \omega(f(n))$ 若 $\lim_{n\to\infty} \frac{g(n)}{f(n)} = \infty$

比值极限法（万能套路）：极限为非零常数 $\to \Theta$；极限为 $0 \to o$（同时成立 O）；极限为 $\infty \to \omega$（同时成立 Ω）。题目要求 “strongest relation” 时只给最强的符号。

增长率层级（必背）：常数 $< \log^\gamma n$（任何固定 γ）$< n < n\log n < n^2 < n^c < 2^n < n!$。

时间复杂度与计算模型

- **时间复杂度（time complexity）**：算法所需**基本操作（primitive operations）**次数。精确常数依赖机器，故用**渐近时间复杂度（asymptotic time complexity）**。
- **最坏情况复杂度（worst case complexity）★**：若算法对**所有**规模为 n 的输入至多需要 $O(f(n))$ 次基本操作，则说它有时间复杂度 $O(f(n))$。本课程只研究**确定性算法（deterministic algorithms）**。
- **字 RAM 模型（word-RAM model）★**：本课程默认模型。假设可常数时间随机访问数组任意位置；每个**字操作（word operation）**（加法、读/写）只需常数时间。数值问题中若中间数指数膨胀，字操作假设失效，应考虑**位复杂度（bit-complexity）**，但本课程中两者差异至多 $\log n$ 因子。
- **多项式时间算法（polynomial time algorithm）**：运行时间为 $O(n^c)$（c 为常数）。注：$10^{10^{10}} n^{100}$ 按渐近标准仍是“好”算法——说明问题有内在结构，区别于暴力枚举最优的问题。
- **归约（reduction）**：既用于设计算法，也用于证明困难性（hardness）。3SUM 可作为“困难”问题：若问题 A 能更快解则 3SUM 也能更快解 $\to$ A 没有更快算法。

两个动机问题

- **旅行商问题（TSP）**：游历每个顶点至少一次的最小代价回路。朴素 $O(n!)$；DP 可到 $O(2^n)$；**NP-完全（NP-complete）**，大概率无高效算法。
- **中国邮递员问题（Chinese postman problem）**：游历每条边至少一次的最小代价回路。可用图匹配（graph matching）在 $O(n^4)$ 内解决。说明“问题结构决定难度”。

核心公式与算法

渐近例子（幻灯片原文）★：

$$100n^2 = \Theta(n^2), \quad 2n^3 + \frac{n^3}{\log} n = \Theta(n^3), \quad n\log n = O(n^2), \quad \frac{n^3}{\log}\log n = o(n^3), \quad 2^n = o(n^n)$$

3SUM 问题：给定 $n+1$ 个数 $a_1, a_2, ..., a_n$ 和 c，判断是否存在 i, j, k 使 $a_i + a_j + a_k = c$。

Algorithm 1：枚举所有三元组，$O(n^3)$。

Algorithm 2：改写为 $c - a_i - a_j = a_k$。先排序 $a_1 \le a_2 \le ... \le a_n$，枚举数对并用二分查找（binary search）查 $c - a_i - a_j$：

$$O(n \log n + n^2 \log n) = O(n^2 \log n)$$

（前一项 sorting，后一项对每对二分）

Algorithm 3（归约到 2-SUM）★：改写为 $a_i + a_j = c - a_k$。对固定的 k 这是 2-SUM 问题，故 3-SUM 归约为 n 个 2-SUM。若已排序，给定 $b := c - a_k$，可在 $O(n)$ 内判定是否存在 i, j 使 $a_i + a_j = b$。对每个 k 试 $b := c - a_k$，得 $O(n^2)$ 算法。

2-SUM 双指针算法（已排序数组，$O(n)$）★：左指针 $L = 1$、右指针 $R = n$，当 $L \le R$ 时循环：检查 $a_L + a_R$ 是否等于目标 b；若 $a_L + a_R > b$ 则 $R := R - 1$；若 $<$ 则 $L := L + 1$。 时间 $O(n)$：每轮 $R - L$ 严格减 1，至多 $n - 1$ 轮。

典型例题：2-SUM 正确性证明 ★

思路：若 $a_L + a_R > b$，则对一切 $R' \ge R$ 有 $a_L + a_{R'} > b$（数组已排序），故可安全地将 R 左移；$<$ 的情形对称。（幻灯片直言这“不是证明”，真正的证明如下。）

证明：

1. 若不存在 i, j 使 $a_i + a_j = b$，算法显然不会误报。
2. 假设存在 i, j 使 $a_i + a_j = b$（不妨 $i \le j$）。
3. 算法持续到 $L = R$，故必有某次迭代使 $L = i$ 或 $R = j$。
4. WLOG 假设 $L = i$ 先发生且此时 $R > j$（另一情形对称）。当 $R > j$ 时 $a_i + a_R > a_i + a_j = b$，故算法会把 R 减 1。
5. 于是 R 不断减小直到 $R = j$，此时 $a_L + a_R = a_i + a_j = b$ 被发现。∎

例：数组 $[1, 3, 6, 8, 9, 11, 14, 20]$，目标 $b = 19$：左指针 $1 \to 2 \to 3 \to 5$，右指针 $20 \to 14$，找到 $8 + 11 = 19$。

易错提醒

- 考试**不需要写精确代码**（precise code），文字描述算法即可；但**作业更侧重正确性证明**。给分规则：Algorithm 2（排序+二分）至少一半分；Algorithm 3 叙述正确但无正确性证明至少一半分；**算法 + 正确性证明 → 满分**。
- “不检查某些数对”的安全性不能只靠直觉，必须给出形式化论证（如上 WLOG 证明）。
- 记号写法 $n^2 \in O(n^3)$、$n^2 = O(n^3)$、$n^2 \ge \Omega(n)$ 在本课程**全部可接受**。
- 渐近分析忽略常数：$2^{2^{100}} n^2$ 按渐近标准比 n^3 “快”，理解这一张力。

第二章　解递归式（L02, Solving Recurrence）

知识点梳理

- **递归式（recurrence relation）**：算法运行时间满足的递推关系。本章统一假设 n 是 2 的幂（至多把输入翻倍，渐近意义下不影响），且约定 $T(i) \le C_2$ for $i \le C_1$（$C_1 \ge 10$，C_2 为绝对常数），即常数规模问题常数时间可解。
- **递归树（recursion tree）**：把递归展开成树，每层操作数求和。分支因子（branching factor）$= a$，深度（depth）$= \log_b n$，宽度 $= a^{\log_b n} = n^{\log_b a}$。
- 求解三方法：**recursion tree（递归树）**、**guess and check / induction（猜测+归纳验证）**、**Master Theorem（主定理）**。

核心公式

Merge sort 与归纳法示例

$$T(n) = T\left(\left\lceil \frac{n}{2} \right\rceil\right) + T\left(\left\lfloor \frac{n}{2} \right\rfloor\right) + O(n) \quad (\text{当 } n = 2^k \text{ 时}) \quad T(n) = 2T\left(\frac{n}{2}\right) + O(n)$$

递归树：每层 cn 操作，共 $\log_2 n + 1$ 层，总计 $cn(\log_2 n + 1) \Rightarrow O(n \log n)$。

Guess-and-check 正确版本：归纳假设 $T(n) = cn \log_2 n$（c 为 $O(n)$ 中的常数）。归纳步：$T(m) = 2T(\frac{m}{2}) + cm = 2c(\frac{m}{2}) \log_2(\frac{m}{2}) + cm = cm(\log_2 m - 1) + cm = cm \log_2 m$。

Master 定理 ★

若 $T(n) = aT(\frac{n}{b}) + n^c$，其中 $a > 0, b > 1, c \geq 0$ 为常数，则

$$T(n) = \begin{cases} O(n^c) & \text{if } c > \log_b a \\ O(n^c \log n) & \text{if } c = \log_b a \\ O(n^{\log_b a}) & \text{if } c < \log_b a \end{cases}$$

推导要点：每层操作数 $n^c, a(\frac{n}{b})^c, a^2(\frac{n}{b^2})^c, \ldots, a^{\log_b n} = n^{\log_b a}$ 构成公比为 $\frac{a}{b^c}$ 的**几何级数**：

- $\frac{a}{b^c} = 1$：每项相同，和为 $n^c \cdot \log_b n$（merge sort 即此情形）；
- $\frac{a}{b^c} < 1$：递减几何级数，首项主导，和为 $O(n^c)$（隐藏常数依赖于 a, b, c——这就是要求它们为常数的原因）；
- $\frac{a}{b^c} > 1$：递增几何级数，末项主导，和为 $O\left(a^{\log_b n}\right) = O\left(n^{\log_b a}\right)$。

> **Remark**（讲义原话）："It is more important to remember the method than the result. It is easy to derive the result back. Also, there are scenarios that the theorem does not apply but the method still works." ——记住方法比记住结论更重要。

常见递归式速查 ★

- $T(n) = T(\frac{n}{2}) + 1 \Rightarrow O(\log n)$（binary search）
- $T(n) = T(\frac{n}{2}) + n \Rightarrow O(n)$（几何级数）
- $T(n) = T(\sqrt{n}) + 1 \Rightarrow O(\log \log n)$（数层数：第 i 层规模为 $n^{2^{-i}}$，$i = \log \log n$ 时降为 $O(1)$）
- $T(n) = 2T(\frac{n}{2}) + 1 \Rightarrow T(n) \leq n - 1$（猜 cn 会失败，见下）
- $T(n) = T(2\frac{n}{3}) + T(\frac{n}{3}) + n \Rightarrow O(n \log n)$（$\log_3 n$ 个"满"层，每层和为 n）
- $T(n) = T(\frac{n}{2}) + T(\frac{n}{4}) + n \Rightarrow O(n)$（每层 $n, 3\frac{n}{4}, 9\frac{n}{16}$ 几何递减；归纳可证 $T(n) \leq 4n$）
- $T(n) = 2T(n-1) + 1 \Rightarrow O(2^n)$（$n$ 层，2^n 个叶节点）
- $T(n) = T(n-1) + T(n-2) + 1 \Rightarrow O\left(\left(\frac{1+\sqrt{5}}{2}\right)^n\right) = O(1.618^n)$（Fibonacci 型，"更快"即更小）

最大独立集（maximum independent set）应用

给定图 $G = (V, E)$，求最大顶点子集 $S \subseteq V$ 使 S 中任意两顶点无边。朴素枚举需 $\Omega(2^n)$。分支策略：取最大度顶点 v，分 $v \in S$（删 v 及其所有邻居，规模至少减 2）和 $v \notin S$（删 v，规模减 1）两种情形：

$$T(n) \leq T(n-1) + T(n-2) + O(n)$$

严格小于 $O(2^n \cdot n)$。

典型例题

例 1（讲义 Exercises）：解 $T(n) = 4T(\frac{n}{2}) + n$，$T(n) = 3T(\frac{n}{2}) + n$，$T(n) = 2T(\frac{n}{2}) + n^2$（$n$ 为 2 的幂，$T(1) = 1$）。解：$O(n^2)$（$\log_2 4 = 2 > 1$）；$O\left(n^{\log_2 3}\right)$（$\log_2 3 \approx 1.585 > 1$）；$O(n^2)$（$c = 2 > 1$）。直接套 Master。

例 2 ★（讲义原题："What is wrong with the following proof?"） 归纳假设：$T(n) = O(n)$。归纳步：$T(m) = 2T(\frac{m}{2}) + O(m) = 2O(\frac{m}{2}) + O(m) = O(m)$。**错因**：归纳假设必须带**显式固定常数**，O 记号在归纳步中隐藏的常数会逐层放大，不能得出对所有 n 成立的同一界。事实上 $T(n) = 2T(\frac{n}{2}) + n$ 的真解是 $\Theta(n \log n)$（$T(1) = 1$ 时 $T(n) = n \log n + n$），直接证伪了这个"证明"。

例 3：$T(n)=2T(\frac{n}{2})+1$。若猜 $T(n)\le cn$，归纳步得 $T(m)=2c(\frac{m}{2})+1=cm+1$ 不成立；必须用 $T(n)\le n-1$ 才能使归纳成立（$2(\frac{m}{2}-1)+1=m-1$）。说明 guess-and-check 的关键是**假设的形式要选对**，常数项要能被吸收。

易错提醒

- ★ **O 记号不能直接放进归纳假设**（例 2 是典型陷阱）；必须写成带显式常数的形式如 $T(n)\le cn$。
- ★ **猜测的形式决定成败**：$2T(\frac{n}{2})+1$ 用 cn 猜不出来，必须用 $n-1$。
- Master 定理要求 a,b,c 为**常数**；定理不适用时（如非均分子问题）递归树方法依然有效。
- n 假设为 2 的幂只是为了简便（至多把输入翻倍，渐近复杂度不变）。

第三章　分治算法（L03, Divide and Conquer）

3.1 逆序对计数（Counting Inversions）★

- **逆序对（inversion）**：输入 n 个互不相同的数 $a_1,\dots,a_n$，输出满足 $i<j$ 但 $a_i>a_j$ 的数对个数（衡量序列“无序程度”）。
- **跨半逆序对（cross inversion）**：一数在左半、一数在右半的逆序对。关键观察：若两半已分别排好序，对右半每个数 c，与之相关的跨逆序对数恰为左半中大于 c 的数的个数。

初版算法（每层各自 sort）：$T(n)=2T(\frac{n}{2})+O(n\log n)\Rightarrow T(n)=\Theta(n\log^2 n)$（用 recursion tree 验证）。**优化洞察**：sort 是瓶颈且不必要——merge 步骤本来就要排序。最终算法 count-and-sort：$T(n)=2T(\frac{n}{2})+O(n)\Rightarrow T(n)=O(n\log n)$，与 merge sort 相同。

两个思路对比：Idea 1——左半已排序，对右半每个 a_j 用 binary search 数左半大于它的个数，每个 $O(\log n)$，总 $O(n\log n)$，“not too slow but we can do better”；Idea 2——在 merge 两个有序表时顺带计数：把右半的数插入合并表时，就知道左半有多少数大于它，$O(n)$ 完成。

例：$(3,7,2,5,4)$ 有 5 个逆序对：$(3,2),(7,2),(7,5),(7,4),(5,4)$。**例**：$[3,9,2,4\mid 6,1,7,8]$：左半 3 对、右半 1 对；排序后 $[2,3,4,9]$、$[1,6,7,8]$；1 对应 4 个跨对，$6,7,8$ 各对应 1 个（都与 9 配对），跨逆序对总数为 7。

★ **先计数、再排序**：左半内部逆序对必须在排序前用递归计数，排好序后再数跨对——顺序反了会丢失信息。考试若问“为什么不能每层 sort”，答：瓶颈且不必要。

3.2 最大子数组（Maximum Subarray）★

输入 $a_1\dots a_n$，输出使 $\sum_{k=i}^{j}a_k$ 最大的 i,j。

- 朴素：枚举所有 i,j 求和 $\Rightarrow O(n^3)$；固定 i 用部分和一次算出所有 $j\Rightarrow O(n^2)$。
- 分治：最优解要么在左半、要么在右半、要么跨中点（$i\le\frac{n}{2}<j$）。跨中点的子数组可拆成 $[i,\frac{n}{2}]$ 和 $[\frac{n}{2}+1,j]$。

Claim（原话，含 if and only if）：For $i\le\frac{n}{2}<j$, $[i,j]$ is a maximum subarray crossing the mid-point **if and only if** $[i,\frac{n}{2}]$ is a maximum subarray ending at $\frac{n}{2}$ and $[\frac{n}{2}+1,j]$ is a maximum subarray starting at $\frac{n}{2}+1$.

证明（反证法）：设 $[i,j]$（$i\le\frac{n}{2}<j$）为最优跨中点解，若 $[i',\frac{n}{2}]$ 的和大于 $[i,\frac{n}{2}]$，则 $[i',j]$ 的和将大于 $[i,j]$，与 $[i,j]$ 的最优性矛盾。故最优跨中点解的左段必为结束于 $\frac{n}{2}$ 的最大子数组（右段对称）。∎

固定起点/终点的最大子数组可用部分和 $O(n)$ 求出。总复杂度 $T(n)=2T(\frac{n}{2})+O(n)\Rightarrow O(n\log n)$。**例**：$1,-3,4,5,6,-20,5,7,7,-3,9,10,11,-10,3,3$ 的最优解为 $5,7,7,-3,9,10,11$。

3.3 中位数的中位数（Median of Medians）★

- 排序法 $O(n\log n)$；但存在 $O(n)$ 算法。推广问题：输入 $a_1\dots a_n$ 与整数 $k\ge1$，输出第 k 小的数。原因：**median 问题不能化归为自身**（无法递归），而 k-th smallest 可以。

- Quicksort 式划分：选 pivot a_i，秩为 r；$r=k$ 则结束；$r>k$ 在 S_1（小于 a_i）中找第 k 小；$r<k$ 在 S_2（大于 a_i）中找第 $k-r$ 小。
- **好 pivot（good pivot）**：秩满足 $\frac{n}{10}\le r\le 9\frac{n}{10}$（离边界不太近），则 $|S_1|=r-1\le 9\frac{n}{10}$ 且 $|S_2|=n-r\le 9\frac{n}{10}$，规模至少缩小 $\frac{n}{10}$。递归式：$T(n)\le T\left(9\frac{n}{10}\right)+P(n)+cn$，其中 $P(n)$ 为找好 pivot 的时间；若 $P(n)=O(n)$ 则 $T(n)=O(n)$。

Median of medians 三步：
1. 把 n 个数分成 $\frac{n}{5}$ 组，每组 5 个数，时间 $O(n)$；
2. 求每组的中位数，记为 $b_1,b_2,\dots,b_{\frac{n}{5}}$，时间 $O(n)$；
3. 求这 $\frac{n}{5}$ 个中位数的中位数，时间 $T\left(\frac{n}{5}\right)$。

Lemma：设 r 为 median of medians 的秩，则 $3\frac{n}{10}\le r\le 7\frac{n}{10}$。**证明思路**：把每组内部排序，再按各组中位数递增排列各组（**这只是为了分析，算法中并不需要真的排序**）。中位数的中位数大于左上角的所有数、小于右下角的所有数；两角各约有 $3\cdot\left(\frac{n}{10}\right)=3\frac{n}{10}$ 个数 $\Rightarrow 3\frac{n}{10}\le r\le 7\frac{n}{10}$。□

时间复杂度：$P(n)=T\left(\frac{n}{5}\right)+c_1n$。故 $T(n)\le T\left(7\frac{n}{10}\right)+P(n)+c_2n=T\left(7\frac{n}{10}\right)+T\left(\frac{n}{5}\right)+(c_1+c_2)n$。用 L02 递归树法可得 $T(n)=O(n)$（两支之和 $7\frac{n}{10}+\frac{n}{5}=9\frac{n}{10}<n$，几何递减），也可用归纳验证。

- ★ **Median 不能直接递归**：子问题“找中位数”不会变成更小的“找中位数”，必须推广到 k-th smallest（概念辨析题）。
- ★ Pivot 不必恰在中点：只要秩不太靠近边界就能保证规模缩小；这是把“找正中”化归为“找不太偏”的关键思想。
- 讲义明确“We won’t discuss randomized algorithms in this course”（随机 pivot 见 DPV 2.4）。
- Homework 辨析：分组大小 5 不是随意的——改成 3 会使 $T(n)\le T\left(2\frac{n}{3}\right)+T\left(\frac{n}{3}\right)+O(n)=O(n\log n)$ 不再是线性；7 仍可线性但常数不同；$\sqrt{n}$ 会破坏线性。

第四章　分治算法（二）：几何与代数（L04）

本讲主旨：分治在计算几何（Closest Pair）与计算机代数（整数乘法、多项式乘法、矩阵乘法、FFT）上给出了已知最快的算法。核心教训：**仅仅套用分治不会自动加速，合并子问题时必须有巧妙方法**。

4.1 最近点对（Closest Pair）★

输入平面上 n 个点 $(x_1,y_1),\dots,(x_n,y_n)$（假设 x 值互异），输出欧氏距离 $\sqrt{(x_i-x_j)^2+(y_i-y_j)^2}$ 最小的下标对。暴力 $O(n^2)$。

分治框架：用垂直线 L 按 x 值中位数分点集：前 $\lceil\frac{n}{2}\rceil$ 个点放入 Q（左侧），后 $\lfloor\frac{n}{2}\rfloor$ 个点放入 R（右侧）。递归求 Q、R 内最近对，设较小距离为 δ。然后只需判定是否存在距离 $<\delta$ 的**跨越对（crossing pair）**——只需关注与直线 L 的 x 距离在 δ 以内的窄带区域。

Observation 1：每个边长 $\frac{\delta}{2}$ 的正方形 box 中至多有一个点。 证明：若同一 box 有两点，其距离至多 $\sqrt{\left(\frac{\delta}{2}\right)^2+\left(\frac{\delta}{2}\right)^2}=\frac{\delta}{\sqrt{2}}<\delta$，与 Q、R 内最近对距离 $\ge\delta$ 矛盾。∎

Observation 2：每个点只需与两层水平层（horizontal layers）内的点计算距离。 证明：若两点被至少两层水平层隔开，则其距离 $>\delta$，不可能是最近对。∎

合并结论：每个点只需检查至多 11 个其它 box（同层 + 下面两层），结合 Observation 1，每个点至多与 11 个其它点算距离；搜索空间从 $\Omega(n^2)$ 对降到 $O(n)$ 对。

算法（6 步）：
1. 用 x 值中位数找划分线 L。Time: $O(n)$。
2. 递归解 Q 和 R 的最近点对，得 δ。Time: $T\left(\frac{n}{2}\right)$。
3. 线性扫描，删去不在 δ 定义的窄带区域内的点。Time: $O(n)$。
4. 按 y 值非递减排序。Time: $O(n\log n)$。
5. 每个点只与其在该 y 序中后 11 个点算距离。Time: $O(n)$。

6. 返回找到的最小距离。

时间复杂度：$T(n) = 2T(\frac{n}{2}) + O(n \log n) \Rightarrow T(n) = O(n \log^2 n)$（瓶颈在递归内排序）。**优化**：在开始时把点按 y 值排好序并在全算法中复用，则 $T(n) = 2T(\frac{n}{2}) + O(n) \Rightarrow T(n) = O(n \log n)$。同样可预先按 x 排序用于划分（不改善最坏复杂度，但改善实际性能）。

★ naive 递归内排序导致 $O(n \log^2 n)$；必须记住优化：y 排序只做一次、全算法复用 $\rightarrow O(n \log n)$。x 值互异的假设用在了哪里、去掉要改什么——讲义明确留作练习，可能考。

4.2 整数乘法与 Karatsuba 算法 ★

给定两个 $2n$-bit 数 x, y，记 $x = x_1 x_2$，$y = y_1 y_2$（x_1, y_1 高 n 位，x_2, y_2 低 n 位）：

$$x = x_1 \cdot 2^n + x_2, \quad y = y_1 \cdot 2^n + y_2$$

$$xy = (x_1 \cdot 2^n + x_2)(y_1 \cdot 2^n + y_2) = x_1 y_1 \cdot 2^{2n} + (x_1 y_2 + x_2 y_1) \cdot 2^n + x_2 y_2$$

$x_1 y_1 \cdot 2^{2n}$ 只是左移 $2n$ 位，不需要乘法。

朴素分治：递归算 $x_1 y_1, x_1 y_2, x_2 y_1, x_2 y_2$ 四个子问题。$T(n) = 4T(\frac{n}{2}) + O(n) \Rightarrow T(n) = O(n^2)$，**没有优于小学算法**。

Karatsuba 算法：只用三个子问题 $x_1 y_1$、$x_2 y_2$ 和 $(x_1 + x_2)(y_1 + y_2)$。中间项恒等式：

$$(x_1 + x_2) \cdot (y_1 + y_2) - x_1 y_1 - x_2 y_2 = x_1 y_2 + x_2 y_1$$

解出三个子问题后，中间项只需 $O(n)$ 位操作。总复杂度：

$$T(n) = 3T\left(\frac{n}{2}\right) + O(n) \quad \Rightarrow \quad T(n) = O\left(n^{\log_2 3}\right) = O(n^{1.59})$$

小学算法的首次显著改进。

多项式乘法：给定两个 n 次多项式 $A(x) = a_n x^n + a_{n-1} x^{n-1} + \ldots + a_1 x + a_0$ 和 $B(x) = \sum_{i=0}^{n} b_i x^i$，同样思路可在 $O(n^{1.59})$ word operations 内算 $A \cdot B(x)$（细节留作练习）。**快速傅里叶变换（FFT）**：整数乘法和多项式乘法有 $O(n \log n)$ 的算法（见 DPV 2.6）。

4.3 矩阵乘法与 Strassen 算法 ★

两个 $2n \times 2n$ 矩阵 A, B，各分为四个 $n \times n$ 块：

$$AB = \begin{pmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{pmatrix} \begin{pmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{pmatrix} = \begin{pmatrix} A_{11}B_{11} + A_{12}B_{21} & A_{11}B_{12} + A_{12}B_{22} \\ A_{21}B_{11} + A_{22}B_{21} & A_{21}B_{12} + A_{22}B_{22} \end{pmatrix}$$

朴素分治：8 个子问题，加法合并只需 $O(n^2)$ word operations。$T(n) = 8T(\frac{n}{2}) + O(n^2) \Rightarrow O(n^3)$，只是把标准算法写成块形式，**无改进**。

Strassen 算法——魔法公式：

$$AB = \begin{pmatrix} P_5 + P_4 - P_2 + P_6 & P_1 + P_2 \\ P_3 + P_4 & P_1 + P_5 - P_3 - P_7 \end{pmatrix}$$

其中

$$\begin{aligned} P_1 &= A_{11}(B_{12} - B_{22}), & P_2 &= (A_{11} + A_{12})B_{22}, \\ P_3 &= (A_{21} + A_{22})B_{11}, & P_4 &= A_{22}(B_{21} - B_{11}), \\ P_5 &= (A_{11} + A_{22})(B_{11} + B_{22}), & P_6 &= (A_{12} - A_{22})(B_{21} + B_{22}), \\ P_7 &= (A_{11} - A_{21})(B_{11} + B_{12}) \end{aligned}$$

每个子问题只用一次乘法加 $O(n^2)$ 额外操作：

$$T(n) = 7T\left(\frac{n}{2}\right) + O(n^2) \quad \Rightarrow \quad T(n) = O\left(n^{\log_2 7}\right) = O(n^{2.81})$$

后续研究推到 $O(n^{2.37})$；Strassen 可实现，$n \geq 5000$ 时快于标准算法。应用：判定图是否有三角形（triangle）可归约到矩阵乘法。

- ★ Karatsuba 的三个子问题必须是 $x_1y_1, x_2y_2, (x_1+x_2)(y_1+y_2)$——能通过 $O(n)$ 加减法还原出中间项 $x_1y_2 + x_2y_1$ 的那组，不是任意三个。
- ★ Strassen 是 7 个子问题，复杂度 $O\left(n^{\log_2 7}\right) = O(n^{2.81})$，不是 $O(n^3)$。
- bit operations vs word operations 的单位不要混：整数乘法用 bit，多项式/矩阵用 word。
- 辨析题："为什么朴素分治整数乘法/矩阵乘法没有改进?"——因为只是把标准算法换了个形式，合并步骤没有巧妙之处。

第五章　广度优先搜索（L05, BFS）

知识点梳理

- 记 $G = (V, E)$ 为无向图，全讲记 $n = |V|$，$m = |E|$。
- **图表示（graph representations）**：
  - **邻接矩阵（adjacency matrix）**：$n \times n$ 矩阵 A，$A[i,j] = \begin{cases} 1 \text{ if } ij \in E \\ 0 \text{ if } ij \notin E \end{cases}$。
  - **邻接表（adjacency list）**：每个顶点维护其邻居的链表。本课程主要用邻接表：空间取决于边数；邻接矩阵需 $\Theta(n^2)$ 空间。**只有邻接表表示才能设计出 $O(m+n)$ word operations 的算法**。
- **图连通性（graph connectivity）**：两顶点 u, v **连通（connected）**指存在从 u 到 v 的 path；顶点子集 $S \subseteq V$ **connected** 指对所有 $u, v \in S$，u 与 v 连通；图 **connected** 指对所有 $s, t \in V$ 连通；**连通分支（connected component）**是极大的（maximally）连通顶点子集。
- **BFS 树（BFS tree）**：对所有从 s 可达的顶点 v，边 $(v, \text{parent}[v])$ 构成一棵树。
- **距离（distance）**：顶点 v 到 s 的距离为 k，若从 s 到 v 的最短路长度为 k。
- **二部图（bipartite graphs）**：顶点可分成 L, R 两部分，所有边都横跨两部分（内部无边）。**图是二部图 $\Leftrightarrow$ 图中没有奇圈（odd cycle）**（MATH 239 结论，充要条件）。

核心公式与算法

BFS 伪代码 ★

```
Input: G = (V, E), s ∈ V.  Output: 从 s 可达的所有顶点。
Initialization: visited[v] = false for all v ∈ V; queue Q is empty.
enqueue(Q, s). visited[s] = true.
While Q ≠ empty do:
    u = dequeue(Q)
    for each neighbor v of u:
        if visited[v] = false: enqueue(Q, v). visited[v] = true.
```

时间复杂度：每个顶点至多入队一次（仅当 visited$[v]$ = false 时）；顶点出队时 for 循环执行 $\deg(v)$ 次。故总时间

$$O\left(n + \sum_{v \in V} \deg(v)\right) = O(n+m)$$

（无向图中 $\sum \deg(v) = 2m$。）

Lemma（BFS 正确性引理）★

存在从 s 到 v 的 path $\Leftrightarrow$ 算法结束时 visited$[v]$ = true。

- **Proof (⇐)**：对算法步数归纳。若 visited$[v]$ = true 则存在 s 到 v 的路。Base case：开始时只有 visited$[s]$ = true。归纳：若 visited$[v]$ 在当前步、顶点 u 的 for 循环内被置 true，则当时 visited$[u]$ = true（u 已在队列中）；由归纳假设有 s 到 u 的路，加上边 uv 即得 s 到 v 的路。

- **Proof (⇒)**：令 $U = \{$“visited”$[v]$ 被置 true 的顶点集$\}$。要证 s 到 $V \setminus U$ 中任一顶点无路。注意 U 与 $V \setminus U$ 之间**没有边**——否则另一端点会被入队并置 true。反证：若有 s 到 $v \in V \setminus U$ 的路，则该路上必有一条“跨越”集合 U 的边，矛盾。∎

引理的应用：图是否连通（检查是否所有 $\text{visited}[v] = \text{true}$）；含 s 的连通分支；s 到 v 是否有路。

BFS 树与最短路 ★

- 回溯路径：加数组 $\text{parent}[v]$；顶点 v 在顶点 u 的 for 循环内**首次**被访问时，置 $\text{parent}[v] = u$。从 v 出发不断找 parent 直到 s，即得 v 到 s 的路径。
- **BFS 树是树的证明**：设含 s 的连通分支有 n 个顶点。除 s 外每个顶点恰有一条连向 parent 的边；这些边不成圈（parent 总是更早被访问）；共 $n-1$ 条边（s 无 parent），无圈的 $n-1$ 边子图必为树。∎
- **最短路**：用 BFS 树从 v 回溯到 s 的路径**就是**从 s 到 v 的最短路。直觉：s 的邻居先被访问（距离 1 的全部入队），然后距离 2 的在距离 ≥ 3 的之前入队……归纳可得所有最短路距离正确。形象说法：friends before friends of friends before friends of friends of friends…**BFS 的主要特性就是能算从 s 出发的最短路**。
- 含距离的 BFS：初始化加 $\text{distance}[s] = 0$；首次访问 v 时加 $\text{parent}[v] = u$，$\text{distance}[v] = \text{distance}[u] + 1$。

二部图判定（BFS 的应用） ★

观察：给定顶点 s，其所有邻居必须在另一侧，邻居的邻居与 s 同侧，依此类推。 **算法**：

- 设 $L = \{v \in V \mid \text{dist}(s,v) \text{ even}\}$，$R = \{v \in V \mid \text{dist}(s,v) \text{ odd}\}$。
- 若有边两端都在 L 内或都在 R 内，返回 “non-bipartite”。
- 否则返回 “bipartite”，(L, R) 为 bipartition。

设图连通（否则每个分支分别处理）。时间 $O(m+n)$（一次 BFS + 每条边检查一次）。

正确性：

- 报 “bipartite” 时显然正确——(L, R) 确实是 bipartition。
- 报 “non-bipartite” 时：设 WLOG 有边 uv，两端 $u, v \in R$。看 BFS 树 T，令 w 为 u, v 在 T 中的**最近公共祖先（lowest common ancestor）**。$\text{dist}(s,u)$ 与 $\text{dist}(s,v)$ 均为奇数（同奇偶），无论 $\text{dist}(s,w)$ 奇偶，T 上 uw 路与 vw 路的长度之和为偶数。于是 $P_{wu} \cup P_{wv} \cup \{uv\}$ 是**奇圈**。故报 non-bipartite 正确。∎

Remarks：① 这给出了“图是二部图 ⇔ 无奇圈”的**算法化证明**；② 这给出了找无向图奇圈的**线性时间算法**；③ 奇圈是“非二部性”的 **“short proof”** ——远好过“试了所有二部划分都失败”。

易错提醒

- BFS 时间复杂度是 $\boldsymbol{O(n+m)}$，不是 $O(n \cdot m)$；成立的前提是**邻接表**表示（邻接矩阵下扫邻居要 $O(n)$）。“每个顶点至多入队一次”是关键。
- $\text{parent}[v]$ 必须在顶点**首次**被访问时设置；$\text{visited}[v] = \text{true}$ 与 enqueue 要**同时**做（否则同一顶点可能被多次入队）。
- BFS 树回溯得到的是从 v 到 s 的路，反向即 s 到 v 的**最短路**——BFS 的核心特性（区别于 DFS）。
- 二部图判定中 $\frac{L}{R}$ 按 $\text{dist}(s,v)$ **奇偶性**划分；发现**内部边**（两端同在 L 或同在 R）即判 non-bipartite。
- 算法只对**连通图**描述，非连通图要对每个分支跑。连通分支是**极大**（maximally）连通子集—— “极大”二字别丢。

第六章　深度优先搜索（L06, DFS）

知识点梳理

- **深度优先搜索（DFS, depth first search）**：图的另一种基本搜索方法；本质上最自然的是**递归（recursive）**算法；非递归实现对应**栈（stack）**数据结构（BFS 对应队列 queue）。讲义原话：“the two fundamental search methods correspond to two fundamental data structures”。

- **s-t 连通性（s-t connectivity）**：从 s 到 t 是否存在路径。DFS 也能检查 s-t connectivity、求含 s 的连通分量、判断图连通性——但★ **DFS 不能计算最短路距离**（这是 BFS 的主要特征）。
- **DFS 树（DFS tree）**：顶点 v 首次被访问时正在执行 $\text{explore}(u)$，则 u 是 v 的 **parent（父节点）**；所有边 $(v, \text{parent}(v))$ 构成一棵树。**同一图可以有多棵不同的 DFS 树，取决于探索邻居的顺序。**
- **祖先（ancestor）/ 后代（descendant）**：u 比 v 更接近根且 u 在 v 到根的路径上，则 u 是 v 的 ancestor；此时 v 是 u 的 descendant。
- **回边（back edge）**★：非树边 uv，若 u 是 v 的 ancestor 或 descendant，则称为 back edge。
- **开始时间 / 结束时间（starting time / finishing time）**：$\text{start}[u]$ 为 u 首次被访问的时间，$\text{finish}[u]$ 为其探索结束的时间。
- **割点（cut vertex，aka articulation point 或 separating vertex）**★：若 $G - v$ 不连通，则 v 是割点。
- **割边（cut edge，aka bridge）**：若 $G - e$ 不连通，则 e 是割边。
- **early[v]**★：$\text{early}[v] := \min\{\text{start}[v], \min\{\text{start}[w] \mid uw \text{ 是回边，} u\text{为}v \text{ 的后代或 } u = v\}\}$。通俗含义：从以 v 为根的子树出发、沿回边能“爬”到的最高处（离根最近的祖先的 start 时间）。

核心公式与算法

DFS 伪代码 ★

```
[Main program]    visited[v] = false  ∀v ∈ V.    visited[s] = true.   explore(s).

explore(u)      // recursive function explore.
    for each neighbor v of u
        if visited[v] = false
            visited[v] = true.   explore(v).
```

时间复杂度：每个顶点 u 的 $\text{explore}(u)$ **至多被调用一次**；调用时 for 循环执行至多 $\deg(u)$ 次。故

$$O\left(n + \sum_{v \in V} \deg(v)\right) = O(n + m) \text{ word operations}$$

即 DFS 为 $O(n + m)$ **线性时间**。

连通性引理（Lemma）：从 s 到 t 存在路径当且仅当结束时 $\text{visited}[t] = \text{true}$（证明与 BFS 相同，留作练习）。

带时间的 DFS 伪代码

```
[Main program]    visited[v] = false  ∀v ∈ V.   time = 1.   visited[s] = true.   explore(s).

explore(u)      // recursive function explore.
    start[u] = time.     time ← time + 1.
    for each neighbor v of u
        if visited[v] = false
            visited[v] = true.   explore(v).
    finish[u] = time.    time ← time + 1.
```

两个关键性质 ★

Property (back edges)：In an undirected graph, all non-tree edges are back edges.（无向图中所有非树边都是回边。） **证明**（反证法）：假设存在边 uv 但 u 与 v 不是祖先-后代对。WLOG 假设 u 先于 v 被访问。则由于 $uv \in E$，v 会在 u 探索结束前被 explore（$\text{explore}(u)$ 遍历邻居 v 时发现未访问就会递归进入）。于是 u 必然是 v 的祖先，矛盾。□

Property (parenthesis)：区间 $[\text{start}(u), \text{finish}(u)]$ 与 $[\text{start}(v), \text{finish}(v)]$ 要么不相交、要么一个包含另一个；后者恰发生在 u, v 为祖先-后代对时。

时间示例：$a : [1, 12]$；$b : [2, 11]$；$d : [3, 10]$；$f : [8, 9]$；$c : [4, 7]$；$e : [5, 6]$。e 的 $[5, 6]$ 被 c 的 $[4, 7]$ 包含（祖先后代）；f 的 $[8, 9]$ 与 c 的 $[4, 7]$ 不相交（非祖先后代）。

割点判定 ★

Lemma（非根割点判定）：非根顶点 v 是割点当且仅当 v 下方存在一棵子树，其边都不连到 v 的（严格）祖先。 **Lemma（根割点判定）**：DFS 树的根是割点当且仅当根至少有两个孩子。

Claim：v 下方子树 T_i 是 $G-v$ 的连通分量当且仅当不存在一端在 T_i、另一端在 v 的（严格）祖先的边。（证明：由 back edge 性质，非树边都是回边，所以不存在从 T_i 连到 v 下方其他子树的边；双向论证见下。）

非根割点引理证明（由 Claim 逐子树应用）：

- （⇒）若 v 的每个下方子树都有一条边连到 v 的祖先，则每棵子树都连到 complement ⇒ $G-v$ 连通 ⇒ v 不是割点。
- （⇐）若某子树 T_i 无边连到 v 的祖先，则 T_i 由 Claim 是 $G-v$ 的连通分量 ⇒ v 是割点。□
- 根引理的证明留作练习。

判割点线性算法（early 数组用法）★：

- 自底向上（从 DFS 树的叶子到根）计算所有 $\text{early}[v]$，总时间 $O(n+\sum \deg(v)) = O(n+m)$。
- **非根割点判据**：顶点 v 是非根割点 ⇔ 存在子节点 u_i 使 $\text{early}[u_i] \geq \text{start}[v]$；反之若对所有子节点都有 $\text{early}[u_i] < \text{start}[v]$，则 v 不是割点。
- **根顶点**：按根引理处理（≥ 2 个孩子即为割点）。
- early 正确性用归纳法证：base case 叶子 v，$\text{early}[v] =$ 与 v 关联的所有边另一端的 start 最小值，耗时 $O(\deg(v))$；归纳步 $\text{early}[v] =$ 子节点 early 值的最小值与 v 所涉及的所有回边另一端的 start 最小值，耗时 $O(\deg(v))$。

例：图上顶点 a, b, c 是割点，边 ab 是割边。祖先关系例：b 是 e 和 f 的祖先，但 c 既不是 f 的祖先也不是后代（不同分支）。

易错提醒

- ★ **DFS 不能求最短路**——与 BFS 的核心区别；高频对比题。
- ★ **根割点判据特殊**：根是割点 ⇔ ≥ 2 个孩子（不要套用 early 公式判根）。
- ★ $\text{early}[u_i] \geq \text{start}[v]$ 是割点的充分条件（“$\geq$”不是“$>$”）——当恰好等于时，回边只连到 v 本身，删除 v 后子树仍断开，v 仍是割点。
- **“(strict) ancestor”**：回边必须连到 v 的**严格**祖先（v 之上），连到 v 自身不算。
- back edge 性质只对**无向图**成立（有向图还有 forward/cross edge，本课只讲无向）。
- 时间复杂度写成 $O\big(n+\sum_{v\in V}\deg(v)\big) = O(n+m)$——求和下标别写错。
- 遍历邻居的顺序影响 DFS 树的形状；同一图多棵 DFS 树。
- 练习题可能变考题：extend the algorithm to identify all cut edges（用 DFS 识别割边）。

第七章　Lab 精选题型

Lab 1：渐近分析与递推求解

Q1：比较函数对的渐近增长率 ★（期中几乎必有一题）

- (a) $f(n) = n^{300}\log n$ vs $g(n) = n(\log n)^{300}$
- (b) $f(n) = \sum_{i=1}^{n}\log i$ vs $g(n) = n\log n$
- (c) $f(n) = 2^n$ vs $g(n) = n!$
- (d) $f(n) = n^{\log n}$ vs $g(n) = (\log n)^n$
- (e) $f(n) = 3^n$ vs $g(n) = 3^{2n}$
- (f) $f(n) = 2^{n^{0.1}}$ vs $g(n) = (\log n)^{\log n}$

套路：两边取 log 对比；用 Stirling 近似 $n! \approx \left(\frac{n}{e}\right)^n$；求和转积分估计。

Q2：解递推式（设 $T(1)=1$）★

- (a) $T(n) = 5T\left(\frac{n}{3}\right) + n^6 \Rightarrow \Theta(n^6)$（$\log_3 5 \approx 1.46 < 6$，case 3）
- (b) $T(n) = 8T\left(\frac{n}{2}\right) + n^3 \Rightarrow \Theta(n^3\log n)$（$8 = 2^3$，边界情形 case 2）

- (c) $T(n) = nT(n-1) + 1 \Rightarrow T(n) = \Theta(n!)$（展开：超指数）
- (d) $T(n) = \sqrt{n} \cdot T(\sqrt{n}) + n$：换元 $n = 2^m$ 或两边除以 n 化简 $\Rightarrow \Theta(n \log\log n)$

套路：(a)(b) 用 Master 三种情形（注意 (b) 是 $a = b^k$ 的边界情形）；(c) 展开/归纳；(d) 换元法。

Q3：棋盘放车，$O(\log n)$ 个矩形计数查询 ★ $n \times n$ 棋盘已有 $n-1$ 个车（每行每列至多一个）；不能直接看棋盘，只能问“矩形 $\{(i,j) : A \le i \le B \text{且} C \le j \le D\}$ 内有几个车”；用 $O(\log n)$ 个查询找到可放最后一个车的格子。 套路：**二分（binary search）**——先对行区间做计数查询二分定位 count $= 0$ 的行，再在该空行内对列区间二分定位 count $= 0$ 的列；两个二分各 $O(\log n)$。考察决策树下界思想、二分搜索、查询复杂度（query complexity）。

Q4：第 $\lfloor \frac{n}{3} \rfloor$ 小元素，$O(n)$ 操作 ★ 给定 $a_1, ..., a_n$，找第 $\lfloor \frac{n}{3} \rfloor$ 小的数。套路：**线性选择算法**——median-of-medians 的变体（5 个一组分组取中位数再递归），或直接套第 k 小元素的线性算法（$k = \lfloor \frac{n}{3} \rfloor$）。

Lab 2：图连通性与分治

Problem 1：连通图所需最少加边数 ★ 无向图 $G = (V, E)$（邻接表）可能有多个连通分支（maximal subsets），求使图连通最少要加几条边。套路：**BFS/DFS 求连通分量数 c**（$O(n+m)$），答案 $= c - 1$（每加一条边至多合并两个分量）。

Problem 2：多数元素（majority element）分治算法 ★ 数组 $A = (a_1, ..., a_n)$ 的多数元指出现超过 $\lfloor \frac{n}{2} \rfloor$ 次的元素。设计分治算法求之并证明正确性。 套路：数组对半分，递归求左右半的候选多数元（至多各 1 个）；合并时数一数两个候选在全数组的出现次数，超 $\lfloor \frac{n}{2} \rfloor$ 即为答案。关键引理：**整体的多数元必是某半的多数元**。正确性用归纳法证；复杂度用 Master 定理分析。

Lab 3：DFS 找长圈 ★

Q1：设图 G 每个顶点度数至少为 k（$k \ge 2$），设计基于 DFS 的算法找含至少 $k+1$ 个顶点的简单圈。给出算法描述、正确性证明、运行时间分析。 Hint：想想 DFS 树叶子的邻居有什么性质。

标准解法（L06 back-edge 性质最经典的应用，期中很可能以类似形式出现）：

1. 对 G 做一次 DFS 得 DFS 树，取**树中最深的叶子** v。
2. 由 back edge 性质：无向图 DFS 树叶子的所有邻居都是其**祖先**（叶子无子节点，非树边只能是连向祖先的回边）。
3. v 的度 $\ge k \Rightarrow v$ 至少有 k 个祖先邻居；取**离 v 最远的那个祖先** w（start 时间最小的邻居）。
4. 从 w 到 v 的树路径加上边 vw 构成简单圈；树路径上至少有 k 个顶点，加上 $v \Rightarrow$ 圈上至少 $k+1$ 个顶点。
5. 运行时间 $O(n+m)$（一次 DFS + 扫描邻居）。

第八章　Homework 1 解答精要

Problem 1：渐近符号判定 ★

对每对函数证明最强的 $O, \Omega, \Theta, o, \omega$ 关系（比值极限法）：

1. $f(n) = 3n^2 + 4n + 5$ vs $g(n) = 2n^2 + 6n\log n$：$\frac{f}{g} = \frac{3 + \frac{4}{n} + \frac{5}{n^2}}{2 + 6\frac{\log n}{n}} \to \frac{3}{2} \Rightarrow \boldsymbol{f = \Theta(g)}$
2. $f(n) = 2^n$ vs $g(n) = n^3$：$\frac{2^n}{n^3} \to \infty$（指数快于任何固定次幂）$\Rightarrow \boldsymbol{f = \omega(g)}$
3. $f(n) = 1000$ vs $g(n) = \log n$：$\frac{1000}{\log} n \to 0 \Rightarrow \boldsymbol{f = o(g)}$
4. $f(n) = \log^\gamma n$ vs $g(n) = 2n$（固定 $\gamma > 0$）：$\frac{(\log n)^\gamma}{2n} \to 0 \Rightarrow \boldsymbol{f = o(g)}$
5. $f(n) = (n-1)!$ vs $g(n) = n!$：$\frac{(n-1)!}{n!} = \frac{1}{n} \to 0 \Rightarrow \boldsymbol{f = o(g)}$

Problem 2：3SUM 计数 $O(n^2)$ ★

给定互异整数 $a_1, ..., a_n$ 和 $c \in \mathbb{N}$，计数所有 $i < j < k$ 使 $a_i + a_j + a_k = c$。算法：排序得 $a_1 < ... < a_n$，计数器置 0；对每个 $i = 1, ..., n-2$，置 $j = i+1, k = n$，当 $j < k$ 时比较 $s = a_i + a_j + a_k$ 与 c：$s < c$ 则 $j+1$；$s > c$ 则 $k-1$；$s = c$ 则计数器 $+1$，然后 $j+1$ 且 $k-1$。正确性要点：固定 i，若 $s < c$，用 a_j 的剩余数对都不可能成立（a_k 已是剩余最大值）；若 $s > c$，用 a_k 的剩余数对都不可能成立；$s = c$ 时由互异性任一端点都不可能再参与其他解，故双指针可同时内移。每个三元组有唯一的最小排序索引，故无重复计数。

运行时间：每次扫描指针只向内移动 $\Rightarrow O(n)$；总计 $O(n\log n + n\cdot n) = O(n^2)$。套路："先排序、固定一维、双指针夹逼"是 k-SUM 家族标准降维套路；命中时双指针**同时**内移（distinct 保证不丢解）。

Problem 3：递推式求解 ★

1. $T(n) = T(n-10) + n \Rightarrow \Theta(n^2)$。上界：猜 $T(n) \le Cn^2$，$C(n-10)^2 + n \le Cn^2$ 对充分大的 C 成立。下界：展开得 $n + (n-10) + (n-20) + \ldots$，前 $\lceil \frac{n}{20} \rceil$ 项每项至少 $\frac{n}{2} \Rightarrow \Omega(n^2)$。（减法递推，Master 不适用；展开是等差数列和）
2. $T(n) = 9T\left(\frac{n}{3}\right) + n^2 \Rightarrow \Theta(n^2 \log n)$。Master case 2：$a = 9, b = 3$，$n^{\log_b a} = n^2$。
3. $T(n) = T\left(\frac{n}{2}\right) + T\left(\frac{n}{3}\right) + n \Rightarrow \Theta(n)$。上界：猜 $T(n) \le Cn$，则 $C\frac{n}{2} + C\frac{n}{3} + n = \left(5\frac{C}{6} + 1\right)n \le Cn$（$C \ge 6$）。下界：$T(n) \ge n \Rightarrow \Omega(n)$。（分叉系数 $\frac{1}{2} + \frac{1}{3} < 1$，几何级数收敛，根支配）
4. $T(n) = 5T\left(\frac{n}{3}\right) + \log^2 n \Rightarrow \Theta\left(n^{\log_3 5}\right)$。Master case 1：$n^{\log_b a} = n^{\log_3 5} \gg \log^2 n$（$\log_3 5 \approx 1.465$）。
5. $T(n) = 2T(n-2) + 1 \Rightarrow \Theta\left(2^{\frac{n}{2}}\right)$。上界：猜 $T(n) \le C2^{\frac{n}{2}} - 1$（注意带 -1 让常数项抵消），则 $2\left(C2^{\frac{n-2}{2}} - 1\right) + 1 = C2^{\frac{n}{2}} - 1$。下界：$T(n) \ge 2T(n-2) \Rightarrow \Omega\left(2^{\frac{n}{2}}\right)$。

易错：第 1 小题是 $\Theta(n^2)$ 不是 $O(n)$；第 5 小题上界猜测要带 "-1"。

Problem 4：两有序数组中位数 $O(\log n)$ ★

A, B 为长度 n 的有序数组，求合并后 $2n$ 个数的中位数。核心思想：不合并，只找"左半 n 个、右半 n 个"的切割点；$i + j = n$ 把二维选择降为一维二分。边界约定 $A_0 = B_0 = -\infty$，$A_{n+1} = B_{n+1} = +\infty$。算法：$\text{lo} = 0, \text{hi} = n$；每步取 $i = \left\lfloor \frac{\text{lo}+\text{hi}}{2} \right\rfloor$，$j = n - i$：

- 若 $A_i > B_{j+1}$，则 $\text{hi} = i - 1$（从 A 取多了，往左搜）；
- 否则若 $B_j > A_{i+1}$，则 $\text{lo} = i + 1$（i 太小，往右搜）；
- 否则返回 $\frac{\max(A_i, B_j) + \min(A_{i+1}, B_{j+1})}{2}$。

正确性：左半为 $A[1..i] \cup B[1..j]$，右半为 $A[i+1..n] \cup B[j+1..n]$；两数组有序，故只需检查边界交叉不等式 $A_i \le B_{j+1}$ 且 $B_j \le A_{i+1}$。找到后 $\max(A_i, B_j)$ 与 $\min(A_{i+1}, B_{j+1})$ 即两中位数，平均即得。运行时间：每步 $O(1)$ 且切割位置范围减半 $\Rightarrow O(\log n)$。套路："二分位置"类题型代表：把问题转化为"找满足单调谓词的分界点"；写伪代码时别忘了边界约定。

Problem 5：直方图最大矩形（分治 $O(n \log n)$）★

连续单位宽度、整数高度 $1 \le h_1, \ldots, h_n \le n^{100}$ 的建筑，最大化 $(j - i + 1) \cdot \min_{i \le k \le j} h_k$ $(1 \le i \le j \le n)$。例：高度 $4, 6, 5, 4, 3, 1, 5, 4, 3, 7$，最大矩形面积为 $4 \cdot 4 = 16$。算法：分治。对当前子数组 $h[L..R]$，取 $m = \left\lfloor \frac{L+R}{2} \right\rfloor$；最优矩形要么全在左半 $h[L..m]$、要么全在右半 $h[m+1..R]$、要么跨中点，前两者递归。跨中点情形：从 $i = m, j = m + 1$ 开始，维护区间 $h[i..j]$ 的最小高度 H；每步向**较高的**下一栋（h_{i-1} 与 h_{j+1} 中较高者；只有一侧可选时走该侧）扩展，更新 H 与最优面积 $(j - i + 1)H$，直到 $i = L, j = R$。正确性：任取一个跨中点的矩形 $J = [a, b]$，设其最小高度为 H^*。初始 $[m, m+1] \subseteq J$；在区间完全包含 J 之前，左右至少一侧的下一栋仍在 J 内（高度 $\ge H^*$）；我们总选较高者，故加入的栋高度也 $\ge H^*$。当区间首次包含整个 J 时，其最小高度 $\ge H^*$ 且宽度 $\ge J$ 的宽度，面积 $\ge J$ 的面积。故扫描找到最优跨中点矩形。三情形取最大即全局最优。运行时间：跨中点扫描 $O(n)$，故 $T(n) = T(\lfloor \frac{n}{2} \rfloor) + T(\lceil \frac{n}{2} \rceil) + O(n) = O(n \log n)$（Master case 2）。套路：分治万能三分法"左/右/跨中点"；跨中点线性合并是难点；正确性证明用"任取最优解，证明算法能支配它"的论证模式。

第九章　综合速查

9.1 公式速查表

场景	公式 / 结论
渐近记号（极限定义）	O: 比值 $\le c$；Ω: 比值 $\ge c$；Θ: 比值 $= c$；o: 比值 $= 0$；ω: 比值 $= \infty$
Master 定理 $aT\left(\frac{n}{b}\right) + n^c$	$c > \log_b a \Rightarrow O(n^c)$；　$c = \log_b a \Rightarrow O(n^c \log n)$；　$c < \log_b a \Rightarrow O\left(n^{\log_b a}\right)$
几何级数判据	公比 $\frac{a}{b^c}$：$= 1$ 每层等量；< 1 首项主导；> 1 末项主导

常用递归式	$T(\frac{n}{2})+1\to O(\log n)$；$T(\frac{n}{2})+n\to O(n)$；$T(\sqrt{n})+1\to O(\log\log n)$； $2T(\frac{n}{2})+n\to\Theta(n\log n)$；$2T(\frac{n}{2})+1\to\Theta(n)$； $T(\frac{n}{2})+T(\frac{n}{4})+n\to O(n)$；$2T(n-1)+1\to O(2^n)$； $T(n-1)+T(n-2)+1\to O(1.618^n)$
Karatsuba	$T(n)=3T(\frac{n}{2})+O(n)\Rightarrow O(n^{\log_2 3})=O(n^{1.59})$；中间项 $(x_1+x_2)(y_1+y_2)-x_1y_1-x_2y_2$
Strassen	$T(n)=7T(\frac{n}{2})+O(n^2)\Rightarrow O(n^{\log_2 7})=O(n^{2.81})$；$P_1...P_7$ 见第四章
最近点对	Observation 1：$\frac{\delta}{2}$ box 至多一点（$\frac{\delta}{\sqrt{2}}<\delta$）；Observation 2：只查两层；每点 11 个点；$T(n)=2T(\frac{n}{2})+O(n)\Rightarrow O(n\log n)$（$y$ 预排序）
BFS	伪代码见第五章；$O(n+\sum\deg(v))=O(n+m)$；$\text{distance}[v]=\text{distance}[u]+1$；最短路
DFS	伪代码见第六章；$O(n+m)$；back edge 性质；parenthesis 区间性质
割点判定	非根：$\exists u_i$ 使 $\text{early}[u_i]\geq\text{start}[v]$；根：$\geq 2$ 个孩子； $\text{early}[v]:=\min\{\text{start}[v],\min\{\text{start}[w]\mid uw\text{ 是回边}\}\}$
二部图判定	$L=\{v\mid\text{dist}(s,v)\text{ 偶}\}$，$R=\{v\mid\text{dist}(s,v)\text{ 奇}\}$；内部边 $\Rightarrow$ non-bipartite；奇圈 $=P_{wu}\cup P_{wv}\cup\{uv\}$
中位数之中位数	三步：分组→组中位数→中位数的中位数；$3\frac{n}{10}\leq r\leq 7\frac{n}{10}$；$T(n)\leq T(7\frac{n}{10})+T(\frac{n}{5})+O(n)\Rightarrow O(n)$
逆序对	$T(n)=2T(\frac{n}{2})+O(n)\Rightarrow O(n\log n)$；先递归计数、再排序、最后数跨对
最大子数组	Claim（iff）：最优跨中点解 $\Leftrightarrow$ 左段为结束于 $\frac{n}{2}$ 的最大子数组且右段为起始于 $\frac{n}{2}+1$ 的最大子数组；$O(n\log n)$
两有序数组中位数	$i=\lfloor\frac{\text{lo}+\text{hi}}{2}\rfloor, j=n-i$；判据 $A_i\leq B_{j+1}$ 且 $B_j\leq A_{i+1}$；中位数 $=\frac{\max(A_i,B_j)+\min(A_{i+1},B_{j+1})}{2}$；$O(\log n)$
直方图最大矩形	左/右/跨中点三分；跨中点向较高侧扩展维护最小高度；$T(n)=2T(\frac{n}{2})+O(n)=O(n\log n)$
3SUM 计数	排序 + 固定 i + 双指针夹逼；$s<c$ 则 $j+1$，$s>c$ 则 $k-1$，$s=c$ 双指针同内移；$O(n^2)$

9.2 高频题型清单 ★

1. ★ **渐近符号判定**：比值极限法；增长率层级；“strongest relation”只给最强符号（o 强于 O，ω 强于 Ω）。
2. ★ **递推式求解**：先看形式——$aT(\frac{n}{b})+f(n)$ 用 Master；$T(n-c)$ 或异分母用 guess-and-check（猜形式、定常数、上下界分开证）。
3. ★ **2-SUM 双指针 + 正确性证明**：WLOG + 对称情形论证；$O(n)$ 时间界（$R-L$ 每轮减 1）。
4. ★ **分治设计三件套**：算法描述 + 正确性证明 + 复杂度分析；“左/右/跨中点”三分法；跨中点情形单独设计线性算法。
5. ★ **BFS/DFS 伪代码默写**：visited 初始化、enqueue/dequeue、for each neighbor；$O(n+m)$ 分析（每点至多入队/被 explore 一次 + $\sum\deg(v)$）。
6. ★ **BFS 引理**：$\text{visited}[v]=\text{true}\Leftrightarrow s$ 到 v 有路，两个方向（归纳 / 反证+跨越边）。
7. ★ **二部图判定**：$\frac{L}{R}$ 按距离奇偶划分；内部边 $\Rightarrow$ non-bipartite；奇圈构造证明（LCA + $P_{wu}\cup P_{wv}\cup\{uv\}$ 为奇圈）。
8. ★ **DFS 割点**：back edge 性质（反证法）；parenthesis 性质；两引理；$\text{early}[u_i]\geq\text{start}[v]$ 判据；给图手算 DFS 树、start/finish、early 值找割点。

9. ★ **Median of medians**：三步算法；Lemma $3\frac{n}{10} \le r \le 7\frac{n}{10}$ 的图示证明；$T(n) \le T\left(7\frac{n}{10}\right) + T\left(\frac{n}{5}\right) + O(n) = O(n)$。
10. ★ **Karatsuba / Strassen**：完整推导；$P_1 \ldots P_7$ 默写；辨析“为什么朴素分治没加速”。
11. ★ **逆序对 / 最大子数组**：merge-and-count 伪代码；Claim 的反证法证明；$\Theta(n \log^2 n)$ vs $O(n \log n)$ 的瓶颈分析。
12. **Lab 3 长圈**：最深叶子 + back edge 性质 + 最远祖先 $\Rightarrow k+1$ 圈（back edge 性质最经典的应用）。
13. **Lab 2**：连通分量数 $c \Rightarrow$ 加 $c-1$ 条边；多数元分治（“整体多数元必是某半多数元”引理）。
14. **Lab 1**：Master 边界情形 $8T(\frac{n}{2}) + n^3$；换元法；棋盘二分 $O(\log n)$ 查询；线性选择。

9.3 证明模板句式（可直接套用）

- **双指针剪枝**：“若 $a_L + a_R > b$，则对一切 $R' \ge R$ 有 $a_L + a_{R'} > b$（数组已排序），故可安全地将 R 左移。”
- **WLOG 对称论证**：“假设存在解且 $L = i$ 先发生（另一情形对称）……于是另一指针不断收敛直到命中。”
- **分治跨中点支配**：“任取一个跨中点的最优解 J，论证算法在某个时刻得到的区间宽度 $\ge$、高度 $\ge$，从而面积 $\ge$ ——故扫描找到最优跨中点解。”
- **归纳法三段**：base case（常数规模）$\rightarrow$ 归纳假设（带显式常数 C）$\rightarrow$ 归纳步代入并解出 C 的条件（如 $C \ge 6$）。
- **反证法（back edge）**：“假设存在非树边 uv 不是回边，WLOG u 先被访问，则 v 会在 u 探索结束前被 explore，u 必为 v 的祖先，矛盾。”
- **奇圈构造**：“设 WLOG 有边 uv 两端同在 R，令 w 为 BFS 树中 u, v 的 LCA，则 $P_{wu} \cup P_{wv} \cup \{uv\}$ 为奇圈。”

9.4 易错点总览 ★

- O 记号不能直接放进归纳假设；猜测形式决定成败（$2T(\frac{n}{2}) + 1$ 要用 $n-1$ 而非 cn；第 5 小题上界带“-1”）。
- $T(n) = T(n-10) + n$ 的解是 $\Theta(n^2)$（等差数列），不是 $O(n)$。
- Master 定理要求 a, b, c 为常数；定理不适用时递归树方法仍有效。
- 逆序对：先递归计数、再排序、最后数跨对；每层 sort 会导致 $\Theta(n \log^2 n)$。
- Median 不能直接递归，必须推广到 k-th smallest；pivot 不必恰在中点。
- Karatsuba 三个子问题是特定的那组；Strassen 是 7 个子问题、$O(n^{2.81})$；bit vs word 操作单位别混。
- BFS 是 $O(n+m)$ 不是 $O(n \cdot m)$，前提是邻接表；$\text{visited} = \text{true}$ 与 enqueue 同时做；$\text{parent}[v]$ 在首次访问时设置。
- DFS 不能求最短路；根割点判据特殊（≥ 2 孩子）；$\text{early}[u_i] \ge \text{start}[v]$ 用 “$\ge$”；回边须连到**严格**祖先；back edge 性质只对无向图成立。
- 二部图判定只对连通图描述，非连通图对每个分支跑；连通分支是**极大**连通子集。
- 两有序数组中位数：别忘了边界约定 $A_0 = B_0 = -\infty$、$A_{n+1} = B_{n+1} = +\infty$。
- 3SUM 计数命中 $s = c$ 时双指针**同时**内移（distinct 保证不丢解）。
- 考试不需要精确代码，文字描述即可；但算法 + 正确性证明 + 复杂度分析三件套缺一不可（Algorithm 2 至少一半分，算法+证明满分）。

—— 祝考试顺利 ——